

Objektorientierung

Inhalt

1. Was ist Objektorientierung?
2. Klassen, Attribute und Methoden
3. Objekte
4. Vererbung
5. Debuggen
6. Listen

Was ist Objektorientierung?

Bisher prozedurale Programmierung

- Bei einer prozeduralen Programmierung wird das Programm als eine sequentielle Folge von Anweisungen geschrieben
- Beispiel Kochbuch:
 - In einem Rezept werden bei der Zubereitung die Anweisungen hintereinander geschrieben

Objektorientierung

- Bei der Objektorientierung wird das Programm als Sammlung von mehreren Klassen geschrieben. Jede Klasse enthält Attribute und Methoden
- Beispiel Kochbuch:
 - Ein Kochbuch kann als Sammlung von Rezept-Objekten betrachtet werden
 - Die Klasse Rezept besteht aus den Attributen Zutaten und Zubereitung
 - Die Klasse Zutat besteht aus den Attributen Menge und Material

Klassen, Attribute und Methoden

Klassen

- Eine Klasse erkennt man an dem Schlüsselwort „class“
- Der Name einer Klasse beginnt mit einem Großbuchstaben
- Attribute und Methoden befinden sich innerhalb der geschweiften Klammern

- **Syntax:**

- `public class <Name>`
`{`
`...`
`}`

- **Beispiel:**

- `public class Zutat`
`{`
`...`
`}`

Packages

- Klassen werden sinnvoll gruppiert in Packages gespeichert, um
 - sie einfacher wieder zu finden
 - Namenskonflikte zu vermeiden
 - den Zugang kontrollieren zu können (Modifizierer protected)
- Das Package der Klasse wird oben in die Klasse geschrieben
- Das Package beginnt mit einem kleinen Buchstaben

■ Syntax:

```
package <Package>;  
public class <Name>  
{  
    ...  
}
```

■ Beispiel:

```
package entities.cookbook;  
public class Zutat  
{  
    ...  
}
```

Imports

- Um eigene Klassen aus anderen Packages oder Standardklassen zu verwenden, müssen sie zuvor importiert werden
- Häufige Imports sind z.B.
 - `java.io`
 - `java.util`
 - `java.awt`
 - `javax.swing`
- Der Import steht in der Klasse unter dem Package und über dem Klassennamen

■ Syntax:

- `package <Package>;
import <Package>.*;
public class <Name>
{ ... }`
- `package <Package>;
import <Package>.<Klassename>;
public class <Name>
{ ... }`

Attribute

- Attribute sind Variablen, die außerhalb von Methoden deklariert sind
- Sie beginnen mit einem Modifizierer

- **Syntax:**

- `<Modifizierer><Datentyp><Name>;`

- **Beispiele:**

- `private int zahl;`
 - `public String wort;`

Getter und Setter

- Getter und Setter sind Methoden die verwendet werden, um Attribute innerhalb einer Methode auszulesen oder zu überschreiben

- **Beispiel:**

- `private int number;`
- `public int getNumber()
{
 return number;
}`
- `public void setNumber (int num)
{
 number=num;
}`

Methoden überladen

- Methoden können überladen werden
- Das heißt, dass der Methodename in einer Klasse mehrfach verwendet werden kann, solange sich die Signatur unterscheidet
 - Die Parameter müssen anders sein (andere Datentypen und/oder mehr/weniger Parameter)
 - Der Rückgabebetyp kann anders sein

Klassen, Attribute und Methoden - Beispiel

- Eine Klasse Zutat soll zwei Attribute erhalten. Das Attribut „menge“ soll ein int sein, das Attribut „material“ ein String. Die Methode „getZutat()“ soll die beiden Attribute in einer Zeile in die Console ausgeben

Klassen, Attribute und Methoden - Beispiel

- Eine Klasse Zutat soll zwei Attribute erhalten. Das Attribut „menge“ soll ein int sein, das Attribut „material“ ein String. Die Methode „getZutat()“ soll die beiden Attribute in einer Zeile in die Console ausgeben

- **Beispiel:**

```
public class Zutat
{
    public int menge;
    public String material;

    public void getZutat()
    {
        System.out.println(menge + " " + material);
    }
}
```

Modifizierer

Modifizierer	Bedeutung
private	Nur innerhalb der Klasse sichtbar
protected	Nur innerhalb abgeleiteter Klassen sichtbar und package-private
public	Öffentlich
ohne	Package-private, d.h. nur innerhalb eines Paketes sichtbar

- man verwendet immer den Modifizierer, der möglichst geschützt ist

Objekte

Objekte erzeugen

- **Syntax:**

- `<Objektyp> <Name> = new <Objektyp>();`

- **Beispiel:**

- `Zutat z = new Zutat();`

- Ein Objekt ist eine Instanz einer Klasse

- In dem Objekt können die Attribute und Methoden der Klasse verwendet werden

Objekte - Attribute und Methoden verwenden

- Durch den Punktoperator kann man vom Objekt auf nicht private Attribute und Methoden zugreifen

- **Syntax:**

- `<Name>.<Attributname>`
- `<Name>.<Methodenname>`

- **Beispiel:**

- `z.menge=5;`
- `z.material=„Eier“;`
- `z.getZutat();`

Konstruktoren

- Ein Konstruktor hat den gleichen Namen und Modifizierer wie die Klasse
- Der Konstruktor wird aufgerufen, wenn das Objekt mit „new“ erzeugt wird
- Wenn kein Konstruktor geschrieben wird, wird ein leerer Standardkonstruktor ausgeführt
- Einem Konstruktor können beliebig viele Parameter mitgegeben werden
- Bei eigenen Konstruktoren wird kein Standardkonstruktor mehr generiert
- Konstruktoren können überladen werden

■ Syntax:

- `public <Name>(<beliebige Parameter>)
{...}`

■ Beispiele:

- `public Zutat()
{...}`
- `public Zutat(int menge, String material)
{...}`

■ Beispiel:

- `Zutat eier5 = new Zutat(5, "Eier");`

Vererbung

Vererbung

- Angenommen es gäbe eine Klasse Rezept. Es beinhaltet eine Liste mit Zutaten und die Zubereitung. Nun soll berücksichtigt werden, dass manche Rezepte einen Backofen benötigen
- Es ist möglich, dass Klassen von anderen Klassen erben und so alle öffentlichen Attribute und Methoden verwenden können
- Methoden und Attribute können überschrieben werden

- **Syntax:**

- `<Modifizierer> class <Name abgeleiteter Klasse> extends <Name Basisklasse>`

- **Beispiel:**

- `public class Ofenrezept extends Rezept`

this und super

- Attribute und Methoden aus dem eigenen Objekt oder dem Basisobjekt können gezielt angesprochen werden
- mit „this“ werden Attribute und Methoden aus dem eigenen Objekt angesprochen
- mit „super“ aus der Basisklasse

- **Syntax:**

- this.<Attribut oder Methodename>
- super.<Attribut oder Methodename>

Methoden überschreiben

- Methoden können überschrieben werden
- Die Methode hat in der Eltern- und Kindklasse den gleichen Namen
- Diese implementiert aber eine andere Logik
- Wichtig für polymorphe Aufrufe
- Wenn eine Methode nicht überschrieben wird in einer Kindklasse, wird die nächstgelegene Implementierung der Methode einer Elternklasse aufgerufen.
- Mit `@Override`-Annotation kann man die Gültigkeit überprüfen

Abstrakte Klassen

- Abstrakte Klassen sind eine Sonderform der Klasse
- Abstrakte Klassen können nicht implementierte Methoden enthalten (Methodensignaturen)
 - In Form von:
public **abstract** <Rückgabetyp> <Methodenname>(<Parameter>);
 - In einer abgeleiteten Klasse müssen sie implementiert werden
- Von abstrakten Klassen kann kein Objekt erzeugt werden

■ **Syntax:**

- public abstract class <Name>

Interfaces (Schnittstellen)

- Schnittstellen sind Spezialfälle von abstrakten Klassen
- Sie enthalten ausschließlich Methodensignaturen, keine Attribute und keine implementierten Methoden

- **Syntax Interface:**

- `public interface <Schnittstellename>`

- **Syntax Klasseneinbindung:**

- `public class <Klassenname> implements <Schnittstellename>`

Enums (Aufzählungstyp)

- Durch Enums können „intelligente“ Konstanten bereitgestellt werden
- Konvention
 - Die einzelnen Enums werden komplett in Großbuchstaben geschrieben
- Die Enum-Klasse kann Attribute, Konstruktoren, Methoden usw. enthalten

Enums (Aufzählungstyp)

- **Syntax:**

- `<Modifizierer> enum <Enumname>`
 {
 <Enum 1>, <Enum 2>, ..., <Enum n>
 }

- **Beispiel:**

- `public enum WochentagEnum`
 {
 MONTAG, DIENSTAG, MITTWOCH, DONNERSTAG, FREITAG,
 SAMSTAG, SONNTAG
 }

Enums (Aufzählungstyp) - Beispiel

- Es soll ein Enum „WochentagEnum“ geschrieben werden, der über einen Konstruktor einen int „number“ für die Reihenfolge erhält

Enums (Aufzählungstyp) - Beispiel

- Es soll ein Enum „WochentagEnum“ geschrieben werden, der über einen Konstruktor einen int „number“ für die Reihenfolge erhält

- **Beispiel:**

```
■ public enum WochentagEnum
{
    MONTAG(1), DIENSTAG(2), MITTWOCH(3), DONNERSTAG(4), FREITAG(5),
    SAMSTAG(6), SONNTAG(7)

    int number;

    WochentagEnum(int number)
    {
        this.number=number;
    }
}
```

Debuggen

Listen

Listen

- Können alle Datentypen oder Objekte abspeichern
- Es existieren verschiedene Listentypen
 - ArrayList (internes Array)
 - LinkedList (verkettete Liste)
- Müssen importiert werden
 - `import java.util.LinkedList;`
 - `import java.util.ArrayList;`
- Sie unterscheiden sich in ihrer Performance
 - Hinzufügen und entfernen bei LinkedList schneller
 - Element erhalten bei ArrayList schneller
- Alle Operationen auf Listen werden mit Methoden ausgeführt

Initialisierung

- **Syntax:**

- `ArrayList <Name> = new ArrayList();`
- `LinkedList <Name> = new LinkedList();`

- **Beispiel:**

- `ArrayList liste = new ArrayList();`

- Bei diesem Aufbau können die Listen alle Datentypen (gleichzeitig) aufnehmen

Initialisierung - generisch

■ Syntax:

- `ArrayList<<Datentyp>> <Name> = new ArrayList<<Datentyp>>();`
- `LinkedList<<Datentyp> <Name> = new LinkedList<<Datentyp>>();`

■ Beispiele:

- `ArrayList<String> namen = new ArrayList<String>();`
- `LinkedList<Integer> zahlen = new LinkedList<Integer>();`

- bei generischen Listen wird der Datentyp der Elemente festgelegt



Als Datentyp können keine primitiven Datentypen genommen werden.

Daher z.B. „Integer“ statt „int“;

Listen - wichtigste Methoden

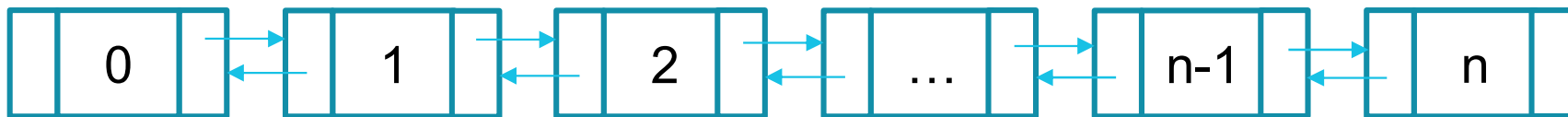
Rückgabe	Name	Input	Funktion
boolean	add	Object	Fügt Element ans Ende der Liste hinzu
Object	get	int	Gibt Element an bestimmter Stelle
boolean	remove	Object	Gibt an, ob das Element in der Liste existiert hat und entfernt es
Object	remove	int	Gibt Element an bestimmter Stelle und entfernt es aus der Liste
int	size		Gibt Anzahl Elemente in Liste

Aufbau ArrayList

- hat ein internes Array gespeichert auf das mit den Methoden zugegriffen wird
- das Array hat eine feste Größe und braucht einen festen Speicherplatz

Aufbau LinkedList

- Verkettete Listen können unendlich wachsen und brauchen nicht mehr Speicherplatz als gespeicherte Elemente
- Elemente werden unzusammenhängend im Speicher abgelegt
- Die Liste besteht aus Knoten
- Jeder Knoten enthält ein Element und einen Zeiger für das nächste und letzte Element



Performance Zugriff

ArrayList

- Direkter Zugriff auf einen beliebigen Index im Array möglich
- Konstante Geschwindigkeit

Performance Zugriff

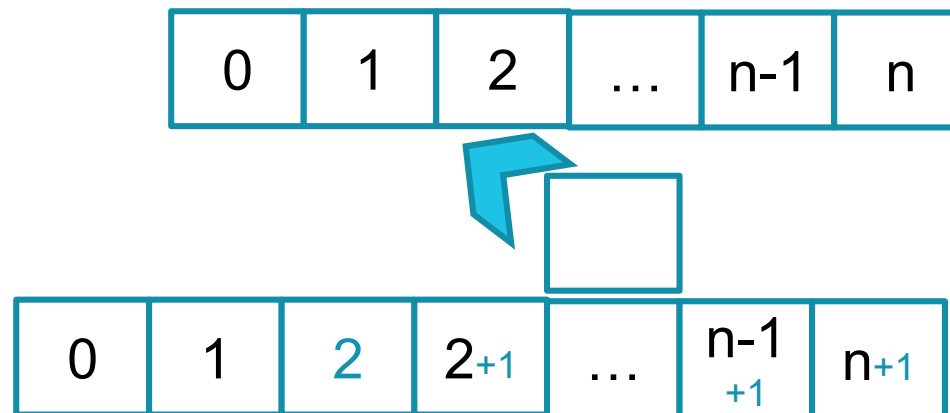
LinkedList

- Zeiger müssen der Reihe nach durchlaufen werden
- Die Laufzeit erhöht sich um so höher der Index ist

Performance Löschen und Einfügen

ArrayList

- Am Ende der Liste problemlos möglich
- Bei Einfügen oder Löschen von Elementen am Anfang oder der Mitte der Liste müssen alle anderen Elemente entsprechend verschoben werden
- Laufzeit erhöht sich um so mehr Elemente verändert werden müssen



Performance Löschen und Einfügen

LinkedList

- Bei Einfügen oder Löschen eines Elements müssen nur die Zeiger entsprechend verändert werden
- Konstante Geschwindigkeit

